



DIVIDE AND CONQUER: PROBLEM SET

MARZIEH ESKANDARI

NJIT

1. You are given an integer array, find its subarray with maximum sum.

0	1	2	3	4	5	6
-8	2	1	0	6	-4	3

- Naïve Approach: Quadratic
- D&C: $O(n \log n)$
- The best approach: linear-time algorithm called **Kadane's Algorithm** (Good to know for interviews).

Naive Approach: Check all possible subarrays

0	1	2	3	4	5	6
-8	2	1	0	6	-4	3

	0	1	2	3	4	5	6
0	-8	-6	-5	-5	1	-3	0
1		2	3	3	9	5	8
2			1	1	7	3	6
3				0	6	2	5
4					6	2	5
5						-4	-1
6							3

D&C Approach:

0	1	2	3	4	5	6
-8	2	1	0	6	-4	3
low			high			

- Split the array into two halves:
- For any subarray, the maximum sum is in one of three places:
 1. Entirely in the **left half**
 2. Entirely in the **right half**
 3. A **crossing subarray** (goes through the middle)
- We compute all three and take the maximum.

D&C Approach:

0	1	2	3		4	5	6
-8	2	1	0		6	-4	3
low					high		

`maxsub(low, high)`

0	1	2	3		4	5	6
-8	2	1	0		6	-4	3

`maxsub(low, mid)`

`maxsub(mid+1, high)`

0	1	2	3		4	5	6
-8	2	1	0		6	-4	3

`middlemaxsub(low, high)`

[crossing subarray]

D&C Approach:

0	1	2	3		4	5	6
-8	2	1	0		6	-4	3
low					high		

0	1	2	3		4	5	6
-8	2	1	0		6	-4	3

0	1	2	3		4	5	6
-8	2	1	0		6	-4	3

maxsub(low,high)

maxsub(low, mid)
maxsub(mid+1, high)

middlemaxsub(low, high)

```

maxsub(low, high):
    if low == high:
        return low, high, A[low]
    mid = (low + high) // 2
    Lstart, Lend, Lsum = maxsub(low, mid)
    Rstart, Rend, Rsum = maxsub(mid+1, high)

    Mstart, Mend, Msum = middlemaxsub(low, high)

    if Lsum >= Rsum and Lsum >= Msum:
        return Lstart, Lend, Lsum
    elif Rsum >= Lsum and Rsum >= Msum:
        return Rstart, Rend, Rsum
    else:
        return Mstart, Mend, Msum

```

```

middlemaxsub(low, high):
    mid=(low+high)//2
    sum1 = -infinity
    sum = 0
    index1 = -1
    for i=mid to low:
        sum += A[i]
        if sum > sum1:
            sum1 = sum
            index1 = i
    sum2 = -infinity
    sum = 0
    index2 = -1
    for i=mid + 1 to high:
        sum += A[i]
        if sum > sum2:
            sum2 = sum
            index2 = i
    return index1, index2, sum1 + sum2

```

2. Given a sorted $n \times n$ integer matrix A , where each row and each column is sorted in increasing order, and an integer q , determine whether q appears in the matrix and output **True** if it is found and **False** otherwise.

q

left

right

top

bottom

	0	1	2	3	4	5	6
0	10	14	20	25	30	45	55
1	12	16	30	45	55	57	60
2	25	28	55	58	60	62	68
3	40	46	60	65	66	68	70
4	45	50	64	68	70	73	75
5	55	57	66	70	72	76	80
6	60	62	70	72	75	80	85

1. Go to middle column: midCol
2. $r = \text{find largest row such that } A[r][\text{midCol}] \leq q$, use binary search

q=50

left

midCol

right

	0	1	2	3	4	5	6	
top	0	10	14	20	25	30	45	55
	1	12	16	30	45	55	57	60
	2	25	28	55	58	60	62	68
	3	40	46	60	65	66	68	70
	4	45	50	64	68	70	73	75
	5	55	57	66	70	72	76	80
bottom	6	60	62	70	72	75	80	85

1. Go to middle column: midCol
2. $r = \text{find largest row such that } A[r][\text{midCol}] \leq q$, use binary search

q=50

top

r

bottom

	left			midCol			right
	↓			↓			↓
	0	1	2	3	4	5	6
0	10	14	20	25	30	45	55
1	12	16	30	45	55	57	60
2	25	28	55	58	60	62	68
3	40	46	60	65	66	68	70
4	45	50	64	68	70	73	75
5	55	57	66	70	72	76	80
6	60	62	70	72	75	80	85

r=1

1. Go to middle column: midCol
2. $r = \text{find largest row such that } A[r][\text{midCol}] \leq q$, use binary search

NOTE: If there is no element in midCol that is less than or equal to q (all values are **greater than q**) then we define $r = \text{top} - 1$.

Example: $q=20$: $r=-1$

	left			midCol			right
	↓			↓			↓
	0	1	2	3	4	5	6
top							
→							
0	10	14	20	25	30	45	55
1	12	16	30	45	55	57	60
2	25	28	55	58	60	62	68
3	40	46	60	65	66	68	70
4	45	50	64	68	70	73	75
5	55	57	66	70	72	76	80
6	60	62	70	72	75	80	85
bottom							
→							

Matrix_search(q, top, r, midCol+1, right)

Matrix_search(q, r+1, bottom, left, midCol-1)

q=50

left

midCol

right

	0	1	2	3	4	5	6	
top	0	10	14	20	25	30	45	55
r	1	12	16	30	45	55	57	60
	2	25	28	55	58	60	62	68
	3	40	46	60	65	66	68	70
	4	45	50	64	68	70	73	75
	5	55	57	66	70	72	76	80
bottom	6	60	62	70	72	75	80	85

```
Matrix_search(q, top, bottom, left, right):
```

```
if top > bottom or left > right:
```

```
    return False
```

```
    midCol = (left + right) // 2
```

```
    # binary search on midCol
```

```
    r = find largest row in range [top..bottom
```

```
    if A[r][midCol] == q:
```

```
        return True
```

```
    result1 = Matrix_search(q, top, r, midCol+1, right)
```

```
    result1 = Matrix_search(q, r+1, bottom, left, midCol-1)
```

```
    return result1 OR result2
```

```
find_largest_row(top, bottom, midCol, q):
```

```
    low = top
```

```
    high = bottom
```

```
    r = top - 1    #no valid row found
```

```
    while low <= high:
```

```
        mid = (low + high) // 2
```

```
        if A[mid][midCol] <= q:
```

```
            r = mid
```

```
            low = mid + 1 else:
```

```
            high = mid - 1 return r
```

3. Given a sorted integer array and an integer q, write pseudocode to find the first (and last) occurrence of q in the array. Use only the Divide and Conquer approach.

0	1	2	3	4	5	6	7	8	9
7	7	9	9	10	10	10	10	11	12

q=10

Output: first=4, last=7

q=7

Output: first=0, last=1

q=11

Output: first=8, last=8

1. Check middle
2. If match → go left
3. Otherwise:
 1. go left if too big
 2. go right if too small
4. Return earliest index found

q=11

0	1	2	3	4	5	6	7	8	9
7	7	9	9	10	10	10	10	11	12

```
FirstOccurrence(A, low, high, q):  
    if low > high:  
        return -1  
    mid = (low + high) // 2  
  
    if A[mid] == q:  
        # continue searching left to find earlier occurrence  
        leftResult = FirstOccurrence(A, low, mid - 1, q)  
  
        if leftResult != -1:  
            return leftResult  
        else:  
            return mid  
  
    elif A[mid] > q:  
        return FirstOccurrence(A, low, mid - 1, q)  
  
    else:  
        return FirstOccurrence(A, mid + 1, high, q)
```



```
LastOccurrence(A, low, high, q):
```

```
    if low > high:  
        return -1
```

```
    mid = (low + high) // 2
```

```
    if A[mid] == q:  
        # continue searching right to find later occurrence  
        rightResult = LastOccurrence(A, mid + 1, high, q)
```

```
        if rightResult != -1:  
            return rightResult  
        else:  
            return mid
```

```
    elif A[mid] > q:  
        return LastOccurrence(A, low, mid - 1, q)
```

```
    else:  
        return LastOccurrence(A, mid + 1, high, q)
```

4. Given a sorted array of distinct non-negative integers, find the smallest integer that does not appear in this array.

0	1	2	3	4	5	6	7	8	9
0	1	2	4	6	7	8	9	11	12

4. Given a sorted array of distinct non-negative integers, find the smallest integer that does not appear in this array.

1. Check middle

0	1	2	3	4	5	6	7	8	9
0	1	2	4	6	7	8	9	11	12

2. If greater than its index: go left,
Otherwise, go right

0	1	2	3	4	5	6	7	8	9
0	1	2	4	6	7	8	9	11	12

4. Given a sorted array of distinct non-negative integers, find the smallest integer that does not appear in this array.

1. Check middle

0	1	2	3	4	5	6	7	8	9
0	1	2	4	6	7	8	9	11	12

2. If greater than its index: go left,
Otherwise, go right

0	1	2	3	4	5	6	7	8	9
0	1	2	4	6	7	8	9	11	12

4. Given a sorted array of distinct non-negative integers, find the smallest integer that does not appear in this array.

1. Check middle

0	1	2	3	4	5	6	7	8	9
0	1	2	4	6	7	8	9	11	12

2. If greater than its index: go left,
Otherwise, go right

0	1	2	3	4	5	6	7	8	9
0	1	2	4	6	7	8	9	11	12

4. Given a sorted array of distinct non-negative integers, find the smallest integer that does not appear in this array.

1. Check middle

0	1	2	3	4	5	6	7	8	9
0	1	2	4	6	7	8	9	11	12

2. If greater than its index: go right,
Otherwise, go right

0	1	2	3	4	5	6	7	8	9
0	1	2	4	6	7	8	9	11	12

4. Given a sorted array of distinct non-negative integers, find the smallest integer that does not appear in this array.

1. Check middle

0	1	2	3	4	5	6	7	8	9
0	1	2	4	6	7	8	9	11	12

2. If greater than its index: go left,
Otherwise, go right

0	1	2	3	4	5	6	7	8	9
0	1	2	4	6	7	8	9	11	12

4. Given a sorted array of distinct non-negative integers, find the smallest integer that does not appear in this array.

```
Missingsmallest(A, low, high):  
  
    if low == high:  
        return low  
  
    mid = (high + low) // 2  
  
    if A[mid] > mid:  
        return Missingsmallest(A, low, mid - 1)  
    else:  
        return Missingsmallest(A, mid + 1, high)
```

ANY QUESTION??

HAVE A NICE DAY!